

GATE CSE Question Bank

Recursion & C Programming

Previous Year GATE Questions with Verified Answers and Step-by-Step Explanations

Contents

- | |
|---|
| • Section A: Recursive Function Tracing |
| • Section B: Pointers, Arrays & Memory |
| • Section C: Static Variables & Scope |
| • Section D: Control Flow & Switch-Case |
| • Section E: Time Complexity & Algorithms |
| • Section F: Data Structures & Theory (MCQ) |
| • Section G: Function Calls, Evaluation Order & I/O |
| • Section H: Matrix Manipulation & Reference Parameters |

Section A: Recursive Function Tracing

Q1 | GATE CS 2011

What is the value of $f(5)$ for the following recursive C function?

```
int f(int n) {
    static int r = 0;
    if (n <= 0) return 1;
    if (n > 3) {
        r = n;
        return f(n - 2) + 2;
    }
    return f(n - 1) + r;
}
```

- (A) 5
- (B) 7
- (C) 9
- (D) 18

Answer: 18

Explanation

$f(5)$: since $n > 3$, r becomes 5 and it returns $f(3) + 2$. $f(3)$: returns $f(2) + r = f(2) + 5$. $f(2)$: returns $f(1) + 5$. $f(1)$: returns $f(0) + 5$. $f(0)$: $n \leq 0$, returns 1. Substituting back: $f(1) = 1 + 5 = 6$, $f(2) = 6 + 5 = 11$, $f(3) = 11 + 5 = 16$, $f(5) = 16 + 2 = 18$.

Q2 | GATE 2011 (2 marks)

What is the return value of the following recursive C function when called as `foo(345, 10)`?

```
unsigned int foo(unsigned int n, unsigned int r) {
    if (n > 0)
        return (n % r + foo(n / r, r));
    else
        return 0;
}
```

- (A) 345
- (B) 12
- (C) 5
- (D) 3

Answer: 12 (Option B)

Explanation

With $r=10$, $n \% r$ peels off the last decimal digit of n on each call, and n / r drops it. So `foo(345, 10)` sums the decimal digits of 345: $3 + 4 + 5 = 12$.

Q3 | GATE 2015 (2 marks)

Consider the following C function. What is printed when it is called as `foo("ABCD EFGH")`?

```
void foo(char *a) {
    if (*a && *a != ' ') {
        foo(a + 1);
        putchar(*a);
    }
}
```

Answer: DCBA

Explanation

The function keeps recursing forward through the string until it hits the space (or end of string), then prints characters on the way back up the call stack — i.e. in reverse order. It stops as soon as it reaches the space, so only "ABCD" is ever visited, and it comes out reversed as "**DCBA**"; "EFGH" is never touched.

Q4 | GATE CS 2000

What does the following recursive C function compute, in terms of a, b and c?

```
int Trial(int a, int b, int c) {
    if ((a >= b) && (c < b))
        return b;
    else if (a >= b)
        return Trial(a, c, b);
    else
        return Trial(b, a, c);
}
```

- (A) The maximum of a, b, c
- (B) The minimum of a, b, c
- (C) The middle value of a, b, c
- (D) None of the above

Answer: The middle value of a, b, c (Option C)

Explanation

Each recursive call reorders the three arguments, narrowing things down until the condition $(a \geq b) \ \&\& \ (c < b)$ holds — at which point b is exactly the value that is neither the largest nor the smallest of the three. The function returns the **median** of a, b, and c.

Q5 | GATE CS 2010 (Set 1)

Consider the function below. How many total calls are made to f (including the initial call) while evaluating f(5)?

```
int f(int n) {
    if (n <= 1) return 1;
    return f(n - 1) + f(n - 2);
}
```

Answer: 15

Explanation

Let $T(n)$ be the number of calls needed for $f(n)$. $T(0)=T(1)=1$ (just the base-case call itself), and $T(n) = 1 + T(n-1) + T(n-2)$ for $n > 1$. That gives $T(2)=3$, $T(3)=5$, $T(4)=9$, $T(5)=1+9+5=15$.

Q6 | GATE CSE 2001

Consider the function below, called as `abc("123")` from `main()`. (a) What is printed? (b) If s is a null-terminated string of n characters, how many characters in total does `abc(s)` print?

```
void abc(char *s) {
    if (s[0] == '\0') return;
    abc(s + 1);
    abc(s + 1);
    printf("%c", s[0]);
}
```

Answer: (a) 3323321 (b) $2^n - 1$ characters

Explanation

Each call on a string of length n makes two recursive calls on a string of length $n-1$ before printing one character, giving the recurrence $C(n) = 2 \cdot C(n-1) + 1$ with $C(0)=0$, whose solution is $C(n) = 2^n - 1$. Tracing "123" by hand: abc on a 1-character string "x" just prints x; abc("23") prints "3","3","2"; abc("123") then prints abc("23") output, then abc("23") output again, then '1' — giving "332"+"332"+"1" = "**3323321**", which is $7 = 2^3 - 1$ characters.

Q7 | GATE CSE 2024 (Set 1)

What does the program print when the input from the command line is 1234 followed by a newline character?

```
#include <stdio.h>
void fX();

int main() {
    fX();
    return 0;
}

void fX() {
    char a;
    if ((a = getchar()) != '\n')
        fX();
    if (a != '\n')
        putchar(a);
}
```

Answer: 4321

Explanation

Each recursive call has its own local variable `a` holding the character it just read, and the `putchar` for that call happens only after its recursive call has fully returned. So the characters get printed in the reverse of the order they were read — a classic "reverse input via recursion" trick. Reading 1,2,3,4 and printing on the way back up the stack gives **4,3,2,1**.

Q8 | GATE CSE 2026 (Set 1)

Consider the recursive functions below. What is the smallest positive integer n for which `foo(n)` returns 5?

```
int bar(int n) {
    if (n == 1) return 0;
    else return 1 + bar(n / 2);
}

int foo(int n) {
    if (n == 1) return 1;
    else return 1 + foo(bar(n));
}
```

Answer: 65536 (i.e. 2^{16})

Explanation

$\text{bar}(n)$ counts how many times n must be halved (integer division) to reach 1, so the smallest n with $\text{bar}(n)=k$ is exactly 2^k . foo climbs one level only when its argument first reaches the smallest n that produced the previous foo -value: $\text{foo}=1$ first at $n=1$, $\text{foo}=2$ first at $n=2$, $\text{foo}=3$ first at $n=4$, $\text{foo}=4$ first at $n=16$, and so $\text{foo}=5$ first occurs at $n = 2^{16} = 65536$.

Q9 | GATE CSE 2015

Consider the following C function. The return value of $\text{fun}(5)$ is _____.

```
int fun(int n) {
    int x = 1, k;
    if (n == 1) return x;
    for (k = 1; k < n; ++k)
        x = x + fun(k) * fun(n - k);
    return x;
}
```

- (A) 51
- (B) 42
- (C) 36
- (D) 60

Answer: 51 (Option A)

Explanation

This is the recurrence for the **Catalan numbers**: $\text{fun}(1)=1$, and $\text{fun}(n) = 1 + \sum_{k=1}^{n-1} \text{fun}(k) \cdot \text{fun}(n-k)$ for $n>1$.
Computing step by step: $\text{fun}(1)=1$, $\text{fun}(2)=1+(\text{fun}(1) \cdot \text{fun}(1))=2$,
 $\text{fun}(3)=1+(\text{fun}(1) \cdot \text{fun}(2)+\text{fun}(2) \cdot \text{fun}(1))=1+(2+2)=5$,
 $\text{fun}(4)=1+(\text{fun}(1)\text{fun}(3)+\text{fun}(2)\text{fun}(2)+\text{fun}(3)\text{fun}(1))=1+(5+4+5)=15$,
 $\text{fun}(5)=1+(\text{fun}(1)\text{fun}(4)+\text{fun}(2)\text{fun}(3)+\text{fun}(3)\text{fun}(2)+\text{fun}(4)\text{fun}(1)) = 1+(15+10+10+15) = 51$. (Verified by direct simulation.)

Q10 | Practice Question (Theory)

A certain processor supports only the immediate and the direct addressing modes. Which of the following programming language features cannot be implemented on this processor?

- (A) Pointers
- (B) Arrays
- (C) Records
- (D) Recursive procedures with local variables

Answer: Recursive procedures with local variables (Option D)

Explanation

Immediate addressing gives a constant operand, and direct addressing gives a fixed, compile-time memory address — neither lets the address used by an instruction be computed at run time. Recursive procedures need a new activation record (local variables) for every call, and the address of each instance is determined only at run time, which requires indirect or indexed addressing. Pointers, arrays, and records can all be handled using direct addressing of statically known base addresses combined with compile-time offset arithmetic in simple cases, but recursion fundamentally needs runtime-computed addresses.

Section B: Pointers, Arrays & Memory

Q11 | GATE CSE 2010 — 1 mark

What does the following program print?

```
#include <stdio.h>
void f(int *p, int *q)
{
    p = q;
    *p = 2;
}

int i = 0, j = 1;

int main()
{
    f(&i, &j);
    printf("%d %d\n", i, j);
    getchar();
    return 0;
}
```

- (A) 2 2
- (B) 2 1
- (C) 0 1
- (D) 0 2

Answer: 0 2 (Option D)

Explanation

In `f(&i, &j)`: `p` initially points to `i`, `q` points to `j`. The line `p = q` makes the local pointer `p` now point to `j` as well (`i` is no longer touched through `p`). Then `*p = 2` writes 2 into `j`. Back in `main`, `i` is untouched (still 0) and `j` has become 2. Output: **0 2**.

Q12 | GATE CSE 2010 — 2 marks

Consider the following program. What is printed?

```
#include <stdio.h>
int f(int *a, int n)
{
    if (n <= 0) return 0;
    else if (*a % 2 == 0)
        return *a + f(a + 1, n - 1);
    else
        return *a - f(a + 1, n - 1);
}

int main()
{
    int a[] = {12, 7, 13, 4, 11, 6};
    printf("%d", f(a, 6));
    getchar();
    return 0;
}
```

- (A) -9
- (B) 5
- (C) 15
- (D) 19

Answer: 15 (Option C)

Explanation

$f()$ is a recursive function that adds $f(a+1, n-1)$ to $*a$ if $*a$ is even, and subtracts $f(a+1, n-1)$ from $*a$ if $*a$ is odd. Working from the innermost call outward over the array {12, 7, 13, 4, 11, 6}: f at 6 (even) = $6+0 = 6$; at 11 (odd) = $11-6 = 5$; at 4 (even) = $4+5 = 9$; at 13 (odd) = $13-9 = 4$; at 7 (odd) = $7-4 = 3$; at 12 (even) = $12+3 = 15$. (Verified by direct compilation.)

Q13 | GATE CSE 2011 — 1 mark

What does the following fragment of C program print?

```
char c[] = "GATE2011";
char *p = c;
printf("%s", p + p[3] - p[1]);
```

- (A) GATE2011
- (B) E2011
- (C) 2011
- (D) 011

Answer: 2011 (Option C)

Explanation

$p[3]$ is the character 'E' (ASCII 69) and $p[1]$ is 'A' (ASCII 65), so $p[3]-p[1] = 69-65 = 4$. $p + 4$ moves the pointer 4 characters into the string "GATE2011", landing right at the substring starting from index 4, which is "**2011**". (Verified by direct compilation.)

Q14 | GATE 2011 — 2 marks

Consider the following recursive C function that takes two arguments. What is the return value of the function foo when it is called as $\text{foo}(345, 10)$?

```
int foo(unsigned int n, unsigned int r) {
    if (n > 0)
        return (n % r + foo(n / r, r));
    else
        return 0;
}
```

- (A) 345
- (B) 12
- (C) 5
- (D) 3

Answer: 12 (Option B)

Explanation

This is the same digit-sum pattern seen earlier: $n \% r$ peels off the last decimal digit of n , and n / r removes it, so the recursion sums the decimal digits of 345: $3 + 4 + 5 = 12$.

Section C: Static Variables & Scope

Q15 | GATE 2012 — 2 marks

Consider the following C code segment. What output will be generated by the given code segment?

```
int a, b, c = 0;
void prtFun(void);

main()
{
    static int a = 1;
    prtFun();
    a += 1;
    prtFun();
    printf("\n%d %d", a, b);
}

void prtFun(void)
{
    static int a = 2;
    int b = 1;
    a += ++b;
    printf("\n%d %d", a, b);
}
```

- (A) 3 1 4 1 4 2
- (B) 4 2 6 1 6 1
- (C) 4 2 6 2 2 0
- (D) 3 1 5 2 5 2

Answer: 4 2 6 2 2 0 (Option C)

Explanation

prtFun() has its own static int a = 2 (separate from main's static a) and a local int b = 1. First call: ++b makes b=2, a += b gives a = 2+2 = 4; prints "4 2". Back in main, main's static a (=1) becomes 2 after a += 1. Second call to prtFun(): its static a persists at 4 from the first call; local b is freshly re-initialized to 1, ++b makes b=2, a becomes 4+2 = 6; prints "6 2". Finally main prints its own a (=2) and the global b (=0, never touched, since prtFun's b is a separate local variable): "2 0". Full output: **4 2 6 2 2 0**. (Verified by direct compilation.)

Q16 | GATE 2002

What is the value of j at the end of execution of the following C program?

```
int incr(int i) {
    static int count = 0;
    count = count + i;
    return count;
}

int main() {
    int i, j;
    for (i = 0; i <= 4; i++)
        j = incr(i);
    return 0;
}
```

Answer: 10

Explanation

count is static, so it persists across calls and keeps accumulating: after $i=0,1,2,3,4$ it becomes 0, 1, 3, 6, 10. The last value assigned to j is **10**.

Q17 | GATE 2014 (Set 2) — 1 mark

Consider the C function given below. Which one of the following is TRUE?

```
int f(int j) {
    static int i = 50;
    int k;
    if (i == j) {
        printf("something");
        k = f(i);
        return 0;
    }
    else return 0;
}
```

- (A) The function returns 0 for all values of j .
- (B) The function prints the string "something" for all values of j .
- (C) The function returns 0 when $j = 50$.
- (D) The function will exhaust the runtime stack or run into an infinite loop when $j = 50$.

Answer: D

Explanation

The static variable i is initialized to 50 once and keeps that value across all calls. When $j = 50$, the condition $i == j$ is true, so it prints "something" and calls $f(i)$, i.e. $f(50)$ again. That recursive call again finds $i == j$ ($50 == 50$) and calls $f(50)$ once more, and so on forever — this never terminates and will **exhaust the runtime stack or loop infinitely** when $j = 50$.

Q18 | Theory (GATE-style)

Which of the following statements are CORRECT?

1. Static allocation of all data areas by a compiler makes it impossible to implement recursion.
2. Automatic garbage collection is essential to implement recursion.
3. Dynamic allocation of activation records is essential to implement recursion.
4. Both heap and stack are essential to implement recursion.

- (A) 1 and 2 only
- (B) 2 and 3 only
- (C) 3 and 4 only
- (D) 1 and 3 only

Answer: 1 and 3 only (Option D)

Explanation

(1) TRUE — if the compiler statically allocates all data areas, there is no fresh storage available for each new function call, which makes recursion impossible. (2) FALSE — automatic garbage collection is not essential for recursion; it only matters for managing dynamically allocated heap memory. (3) TRUE — the number of activation records needed for a recursive call chain is not known at compile time, so they must be dynamically allocated (typically on the call stack). (4) FALSE — only the stack is essential for function calls; the heap is used separately for dynamic memory that the programmer explicitly allocates.

Section D: Control Flow & Switch-Case

Q19 | GATE 2012 — 2 marks

What will be the output of the following C program segment?

```
char inchar = 'A';
switch (inchar)
{
    case 'A':
        printf("choice A ");
    case 'B':
        printf("choice B ");
    case 'C':
    case 'D':
    case 'E':
    default:
        printf("No Choice");
}
```

- (A) No Choice
- (B) Choice A
- (C) Choice A Choice B No Choice
- (D) Program gives no output as it is erroneous

Answer: Choice A Choice B No Choice (Option C)

Explanation

Since inchar is 'A', execution starts at case 'A' and prints "choice A ". Because there is no break statement, execution **falls through** into case 'B', printing "choice B ". It keeps falling through 'C', 'D', 'E' (which have no statements) and reaches default, printing "No Choice". Combined output: **"Choice A Choice B No Choice"**.

Section E: Time Complexity & Algorithms

Q20 | GATE 2013 — 2 marks

Consider the following function. The return value of the function is__

```
int unknown(int n) {
    int i, j, k = 0;
    for (i = n / 2; i <= n; i++)
        for (j = 2; j <= n; j = j * 2)
            k = k + n / 2;
    return k;
}
```

- (A) $\Theta(n^2)$
- (B) $\Theta(n^2 \log n)$
- (C) $\Theta(n^3)$
- (D) $\Theta(n^3 \log n)$

Answer: $\Theta(n^2 \log n)$ (Option B)

Explanation

The outer loop runs for roughly $(n/2)$ iterations (from $n/2$ to n), and the inner loop, which doubles j each time, runs about $\log n$ times. So the total number of times the innermost statement executes is approximately $(n/2) \cdot \log n$, and each execution adds $n/2$ to k . The final value of k is therefore proportional to $(n/2) \cdot (n/2) \cdot \log n = \Theta(n^2 \log n)$.

Q21 | GATE 2014 (Set 2) — 2 marks

Give a value q (to 2 decimals) such that $f(q)$ will return q .

```
#include <math.h>

double f(double x) {
    if (fabs(x * x - 3) < 0.01)
        return x;
    else
        return f(x / 2 + 1.5 / x);
}
```

Answer: 1.73

Explanation

The stopping condition is $|x^2 - 3| < 0.01$, so we need q such that $2.99 < q^2 < 3.01$, i.e. q in the range $(1.7292, 1.7349)$ roughly. $\sqrt{3} \approx 1.73205$ lies in this range, and rounding to two decimal places gives **1.73**. Verification: $1.73^2 = 2.9929$, and $|2.9929 - 3| = 0.0071 < 0.01$, so the stopping condition holds and $f(1.73)$ would return 1.73.

Q22 | GATE 2014 (Set 2) — 1 mark

Consider the function shown below. The value returned by `func(435)` is _____.

```
int func(int num) {
    int count = 0;
    while (num) {
        count++;
        num >>= 1;
    }
    return count;
}
```

Answer: 9

Explanation

435 in binary is 110110011, which has 9 bits. The loop runs once for each right-shift needed to bring num down to 0: $435 \rightarrow 217 \rightarrow 108 \rightarrow 54 \rightarrow 27 \rightarrow 13 \rightarrow 6 \rightarrow 3 \rightarrow 1 \rightarrow 0$, which is 9 shifts. So count ends at **9**. (Verified by direct compilation.)

Q23 | GATE CS 2013

An array of 200 elements is to be sorted using merge sort. The number of comparisons required in the worst case is closest to:

- (A) $200 \log 200$
- (B) 200^2
- (C) $200 \log (200^2)$
- (D) 200

Answer: 200 log 200 (Option A)

Explanation

Merge sort performs $\Theta(n \log n)$ comparisons in the worst case. For $n = 200$, that is closest to **200 log 200**.

Q24 | GATE CS 1995

A list of n numbers is given. Which of the following methods can be used to find the maximum and the minimum element together in $O(n)$ time?

- (A) Tournament method
- (B) Divide and conquer
- (C) Dynamic programming
- (D) Greedy algorithm

Answer: Tournament method (Option A)

Explanation

The tournament (pairwise-comparison) method pairs elements up, compares within pairs, then separately tracks the largest of the pair-maximums and the smallest of the pair-minimums, finding both max and min together in about $3n/2$ comparisons — **linear time**.

Q25 | GATE CSE 2025 (Set 1)

Consider the following pseudocode, and let $A[0..29]$ be an array storing 30 distinct integers already arranged in descending order. How many swap operations are performed when $\text{fun}()$ is called on A ?

```
fun(int A[0...n-1]) {
    for i = 0 to n-2
        for j = 0 to n-i-2
            if (A[j] > A[j+1])
                swap A[j] and A[j+1]
}
```

Answer: 435

Explanation

This is one pass structure of bubble sort, which makes $n(n-1)/2$ comparisons in total — for $n=30$ that is $30 \times 29/2 = 435$. Because the array starts in strictly descending order (the worst case for this ascending-sort routine), every single comparison turns out to find elements out of order, so every comparison triggers a swap. Total swaps = **435**.

Section F: Data Structures & Theory (MCQ)

Q26 | GATE CS 2004

A single array $A[1..MAXSIZE]$ is used to implement two stacks. The two stacks grow from opposite ends of the array. Variables $top1$ and $top2$ ($top1 < top2$) mark the topmost element of each stack. If the space is to be used efficiently, the condition for "stack full" is:

- (A) $top1 + 1 = top2$
- (B) $top1 = top2 + 1$
- (C) $top1 = top2 + MAXSIZE$
- (D) $top1 = top2 - MAXSIZE$

Answer: $top1 + 1 = top2$ (Option A)

Explanation

The two stacks grow toward each other from opposite ends of the same array. They run out of room exactly when the gap between the two top pointers closes completely, i.e. when $top1 + 1 = top2$.

Q27 | GATE CSE 2003

Given the C declarations `int *A[10], B[10][10]`; consider:

I. $A[2]$ II. $A[2][3]$ III. $B[1]$ IV. $B[2][3]$

Which of these will NOT give a compile-time error if used as the left-hand side of an assignment statement?

Answer: I, II, and IV (not III)

Explanation

$A[2]$ is a single pointer value — assignable. $A[2][3]$ dereferences that pointer to a single `int` — assignable. $B[2][3]$ is a single `int` element of the 2-D array — assignable. $B[1]$, however, denotes an entire row (an array), and an array name is not a modifiable lvalue — you cannot reassign it as a whole — so **III alone** causes a compile-time error.

Q28 | GATE CS 2005

A program reads in 500 integers in the range $[0, 100]$, representing the scores of 500 students. It then prints the frequency of each score strictly above 50. What is the best way for the program to store the frequencies?

- (A) An array of 50 numbers
- (B) An array of 100 numbers
- (C) An array of 500 numbers
- (D) A dynamically allocated array of 250 numbers

Answer: An array of 50 numbers (Option A)

Explanation

Scores "above 50" means the values 51 through 100 — exactly 50 distinct possible scores — so a 50-element frequency array is enough; allocating for 100 or 500 would waste space. Answer: **an array of 50 numbers**.

Section G: Function Calls, Evaluation Order & I/O

Q29 | GATE CSE 2024 (Set 2)

Consider the following C program. Assume parameters to a function are evaluated from right to left. What is the output?

```
#include <stdio.h>
int g(int p) { printf("%d", p); return p; }
int h(int q) { printf("%d", q); return q; }
void f(int x, int y) { g(x); h(y); }

int main() {
    f(g(10), h(20));
    return 0;
}
```

Answer: 20101020

Explanation

Arguments to f are evaluated right to left, so h(20) runs first (prints "20", returns 20), then g(10) runs (prints "10", returns 10). These results become x=10, y=20 inside f. Inside f, g(x) = g(10) prints "10"; then h(y) = h(20) prints "20". Reading the prints in the order they happen: 20, 10, 10, 20 → **20101020**.

Q30 | GATE CSE 2025 (Set 2)

What is the output of the following C program?

```
#include <stdio.h>
int g(int n) { return (n + 10); }
int f(int n) { return g(n * 2); }

int main() {
    int sum = 0, n;
    for (n = 1; n < 3; n++)
        sum += g(f(n));
    printf("%d", sum);
    return 0;
}
```

Answer: 46

Explanation

For n=1: f(1)=g(2)=12, so g(f(1))=g(12)=22. For n=2: f(2)=g(4)=14, so g(f(2))=g(14)=24. sum = 22 + 24 = **46**.

Q31 | GATE CSE 2025 (Set 1)

What is the output of the following C program?

```
#include <stdio.h>
void foo(int *p, int x) { *p = x; }

int main() {
    int *z;
    int a = 20, b = 25;
    z = &a;
    foo(z, b);
    printf("%d", a);
    return 0;
}
```

Answer: 25

Explanation

z stores the address of a. foo receives that address as p and does *p = x, i.e. it writes the value of x (25) directly into a's memory. So a becomes **25**.

Q32 | GATE CSE 2015 (Set 1)

What is the output of the following C program?

```
#include <stdio.h>
void f1(int a, int b) {
    int c;
    c = a; a = b; b = c;
}
void f2(int *a, int *b) {
    int c;
    c = *a; *a = *b; *b = c;
}

int main() {
    int a = 4, b = 5, c = 6;
    f1(a, b);
    f2(&b, &c);
    printf("%d", c - a - b);
    return 0;
}
```

Answer: -5

Explanation

f1 swaps copies of a and b (call-by-value), so it has no effect on main's a and b; they stay a=4, b=5. f2 swaps through pointers (call-by-reference), so it really swaps b and c: b becomes 6, c becomes 5. Result: c - a - b = 5 - 4 - 6 = **-5**.

Q33 | GATE CSE 2024 (Set 2)

Consider the following C function. What does fX(a) compute, for a null-terminated string a?

```
int fX(char *a) {
    char *b = a;
    while (*b) b++;
    return b - a;
}
```

Answer: Returns the length of string a (a hand-written strlen)

Explanation

b starts at the same address as a and is advanced one character at a time until it reaches the null terminator. The pointer difference b - a then equals exactly the number of characters that were stepped over, which is the string's **length** — this is essentially a hand-written equivalent of strlen().

Q34 | GATE CS 2014 (Set 1)

Consider the following C function, where E is an array of size elements. What does the value returned by MyX represent, in terms of the array E?

```
int MyX(int *E, unsigned int size) {
    int Y = 0, Z, i, j, k;
    for (i = 0; i < size; i++)
        Y = Y + E[i];
}
```



```

    for (i = 0; i < size; i++)
        for (j = i; j < size; j++) {
            Z = 0;
            for (k = i; k <= j; k++)
                Z = Z + E[k];
            if (Z > Y)
                Y = Z;
        }
    return Y;
}

```

Answer: The maximum sum among all contiguous sub-arrays of E

Explanation

Y starts out equal to the sum of the entire array. The nested loops then examine the sum Z of every contiguous sub-array E[i..j], and whenever a sub-array beats the current best, Y is updated. So Y ends up holding the classic "**maximum subarray sum**", which is always at least as large as the sum of the whole array.

Q35 | GATE CS 2016 (Set 2)

What is the output of the following ANSI C program?

```

#include <stdio.h>
int main() {
    int arr[4][4];
    int i, j;
    for (i = 0; i < 4; i++)
        for (j = 0; j < 4; j++)
            arr[i][j] = 10 * i + j;
    printf("%d", *((arr + 2) + 3));
    return 0;
}

```

Answer: 23

Explanation

*(arr+2) is row 2 of the array; adding 3 and dereferencing reaches arr[2][3]. By the fill rule arr[i][j] = 10*i + j, arr[2][3] = 10*2 + 3 = **23**.

Section H: Matrix Manipulation & Reference Parameters

Q36 | GATE 2014 (Set 3) — 2 marks

Let A be a square matrix of size $n \times n$. Consider the following pseudocode. What is the expected output?

```
C = 100;
for i = 1 to n do
  for j = 1 to n do
    {
      Temp = A[i][j] + C;
      A[i][j] = A[j][i];
      A[j][i] = Temp - C;
    }
  for i = 1 to n do
    for j = 1 to n do
      output(A[i][j]);
```

- (A) The matrix A itself
- (B) Transpose of the matrix A
- (C) Adding 100 to the upper diagonal elements and subtracting 100 from lower diagonal elements of A
- (D) None of the above

Answer: Transpose of the matrix A (Option B)

Explanation

For every pair (i, j) , the algorithm temporarily stores $A[i][j]$ (offset by C , which cancels out), sets $A[i][j] = A[j][i]$, and sets $A[j][i] = \text{original } A[i][j]$. This is exactly an in-place **swap of $A[i][j]$ and $A[j][i]$** for every pair, which is precisely how you compute the **transpose** of the matrix.

Q37 | GATE 2013 (C++) — 1 mark

What is the return value of $f(p, p)$, if the value of p is initialized to 5 before the call? Note that the first parameter is passed by reference, whereas the second parameter is passed by value.

```
int f(int &x, int c) {
  c = c - 1;
  if (c == 0) return 1;
  x = x + 1;
  return f(x, c) * x;
}
```

- (A) 3024
- (B) 6561
- (C) 55440
- (D) 161051

Answer: 6561 (Option B)

Explanation

x is a reference to p (so updates to x persist in p across all calls), while c is a fresh copy of p 's value (5) for the outer call, decremented locally each level. Tracing through: outer call has $c=5 \rightarrow 4$, $x(=p)$ becomes 6, then calls $f(x, 4)$; that call has $c=4 \rightarrow 3$, x becomes 7, calls $f(x, 3)$; $c=3 \rightarrow 2$, x becomes 8, calls $f(x, 2)$; $c=2 \rightarrow 1$, x becomes 9, calls $f(x, 1)$; $c=1 \rightarrow 0$, returns 1 immediately (x stays 9). Unwinding and multiplying by the x value at each level: $1 \times 9 \times 8 \times 7 \times 6 = 6561$. (Verified by direct compilation.)

End of Question Bank — All answers verified by direct compilation/execution where applicable.